BRUTE FORCE

ELECTRONICS-PROGRAMMING-ROBOTICS

bruteforcehub@gmail.com +263718384192

Course Overview

- Week 1 – Electronics
- Week 2 – Programming
- Week 3 – Arduino
- Week 4 – Robotics

BRUTE FORCE bruteforcehub@gmail.com +263718384192

Tutorial 4

Remote Control

Focus

- IR Remote working principle
- IR Receiver working principle
- Remote control buttons
- Controlling RGB LED using Remote control
- Controlling LEDs using Remote Control
- Controlling DC Motor using Remote Control
- Controlling servo motor using Remote Control

IR Remote Working Principle…

- An IR remote (also called a transmitter) uses light to carry signals from the remote to the device it controls.
- It emits pulses of invisible infrared light that correspond to specific binary codes.
- IR remotes use LED lights to transmit their infrared signals.

BRUTE FORCE bruteforcehub@gmail.com +263718384192

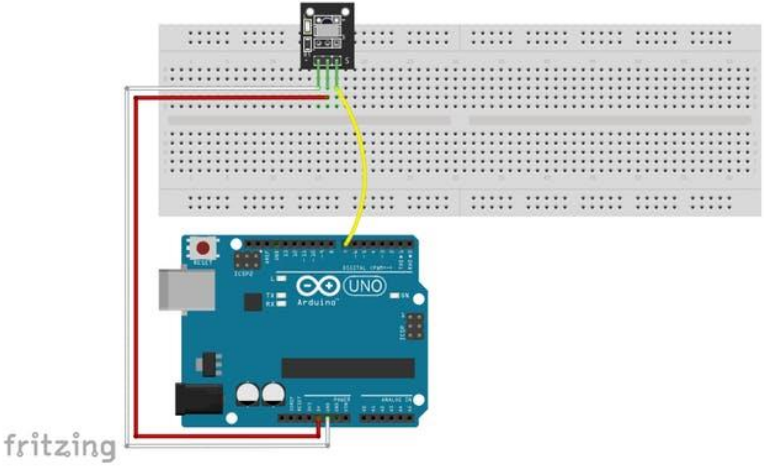
IR receiver working principle

- Infrared light, with a wavelength longer than visible light, is not within the range of human vision.
- When you press a button on your TV control, an LED on your control turns on and off continuously and causes a modulated infrared signal to send from the control to your TV.
- The command will execute after the signal is demodulated.
- IR receiver modules are used to receive IR signals.
- These modules work in 3, 8 KHz frequency. When the sensor is not exposed to any light at its working frequency, the Vout output has a value equal to VS (power supply).
- With exposing to a 38 kHz infrared light, this output will be zero.


1 Signal
2 +V
3 GND

Code for each Remote Control Button

- To set up a connection between the Arduino and an IR sender and receiver.
- To do this, we first need to know the code for each button on the remote control.
- By pressing each button, a specific signal sends to the receiver and will be displayed on the Serial Monitor window.



Coding…

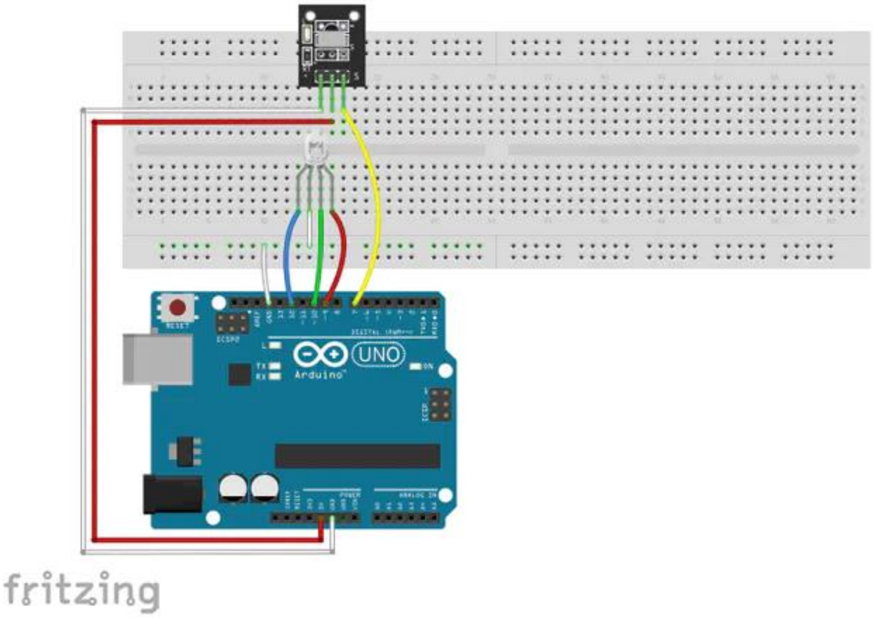
- install the IR library to use an IR module.
- Download the library from the following link and in the Sketch window, open the Include library option and select IRRemote.h.
- This library may be available in your Arduino libraries by default. In this case, you don't need to install it.

```
#include <IRremote.h> //including infrared remote header
file int RECV_PIN = 7; // the pin where you connect the
output pin of IR sensor IRrecv irrecv(RECV_PIN);
decode_results results;
void setup()
{ Serial.begin(9600);
 irrecv.enableIRIn();
{ void loop()
{
if (irrecv.decode(&results))// Returns 0 if no data ready, 1 if
data ready.
{
int results.value = results;// Results of decoding are stored in
result.value Serial.println(" ");
 Serial.print("Code: "); Serial.println(results.value); //prints
the value a a button press
 Serial.println(" ");
irrecv.resume(); // Restart the ISR state machine and Receive
the next value }
```

Controlling an RGB LED Colors Using the IR Remote Control

- After you found the code for each button, you can use it to control the commands.
- In this example, we connected an RGB LED to Arduino and use the remote control to change the colors.
- To do this, specify a few buttons on the remote control and save their code.
- In this example, buttons 1 to 3 are used.
- Then assign a specific color to each button.
- At the end by pressing any of the 1 to 3 keys, the LED changes its color.

Circuit



 bruteforcehub@gmail.com +263718384192

Coding…

```
#include <IRremote.h>
int RECV_PIN =6;
int bluePin = 11;
int greenPin = 10;
 int redPin = 9;
 IRrecv irrecv(RECV_PIN);
 decode_results results;
 void setup()
{ Serial.begin(9600);
irrecv.enableIRIn();
 pinMode(redPin, OUTPUT);
pinMode(greenPin, OUTPUT);
 pinMode(bluePin, OUTPUT);
}
void loop()
{
if (irrecv.decode(&results))
{
int value = results.value;
Serial.println(value);
switch(value){ case 12495: //Keypad
button "1" //set color red
analogWrite(redPin, 0xFF);
analogWrite(greenPin,0x08);
analogWrite(bluePin, 0xFB); }
switch(value){ case -7177: //Keypad
button "2" //set color skyblue
analogWrite(redPin, 0x00);
analogWrite(greenPin,0xFF);
analogWrite(bluePin, 0xFF); }
```

```
switch(value){ case 539: //Keypad button "3" //set color
pink analogWrite(redPin, 0x1F);
analogWrite(greenPin,0x00); analogWrite(bluePin, 0x8F);
} switch(value){ case 25979: //Keypad button "4" //set
color light green analogWrite(redPin, 0x11);
analogWrite(greenPin,0x5F); analogWrite(bluePin, 0x01);
} irrecv.resume(); } }
```

BRUTE FORCE bruteforcehub@gmail.com +263718384192

Controlling LEDs using IR Remote

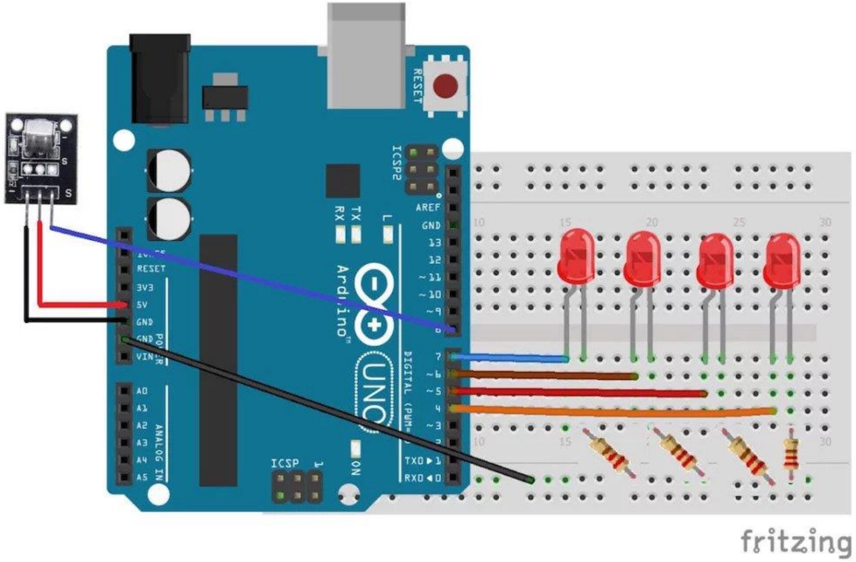
How Does It Work?

- Whenever a button is pressed on the remote, it sends an infrared signal in encoded form.
- This signal is then received by the IR receiver and given to the Arduino.
- We will save the code for the buttons that we want to control the LEDs in the Arduino code.
- Whenever a button on the remote is pressed, the Arduino receives a code.
- The Arduino will compare this code with the codes already saved, and if any of them match, the Arduino will turn on the LED connected to that button.

Circuit Diagram

- Connect the positives of the four LEDs to the pins 7, 6, 5, and 4.
- Connect the negative of the four LEDs to GND on the Arduino through the 220 ohm resistors.
- The longer wires on the LEDs are positive and the shorter wires are negative.
- Connect the negative wire on the IR sensor to GND on the Arduino.
- Connect the middle of the IR sensor which is the VCC to 5V on the Arduino.
- Connect the signal pin on the IR sensor to pin 8 on the Arduino.

Circuitry



Materials

- Arduino Uno/Nano
- Breadboard
- 4 LEDs
- 4 Resistors
- IR transmitter LED
- Power supply

Programming

```
#include
#define first_key 48703
#define second_key 58359
#define third_key 539
#define fourth_key 25979
int receiver_pin = 8;
int first_led_pin = 7;
int second_led_pin = 6;
int third_led_pin = 5;
int fourth_led_pin = 4;
int led[] = {0,0,0,0};
IRrecv receiver(receiver_pin);
decode_results output;
```

```
void setup()
{
Serial.begin(9600);
receiver.enableIRIn();
pinMode(first_led_pin, OUTPUT); pinMode(second_led_pin,
OUTPUT); pinMode(third_led_pin, OUTPUT);
pinMode(fourth_led_pin, OUTPUT);
}
```

Programming

```
void loop()
{
if (receiver.decode(&output))
{ unsigned int value  = output.value;
switch(value)
{
case first_key: if(led[1]  == 1)
{ digitalWrite(first_led_pin,  LOW);
 led[1] = 0;
 }
else
{
 digitalWrite(first_led_pin,  HIGH);
led[1] = 1; }
break;
```

```
case second_key:
if(led[2]  == 1)
 {
digitalWrite(second_led_pin,  LOW);
led[2] = 0;
}
 else
{
 digitalWrite(second_led_pin,  HIGH);
 led[2] = 1;
 }
break;
```


bruteforcehub@gmail.com **+263718384192**

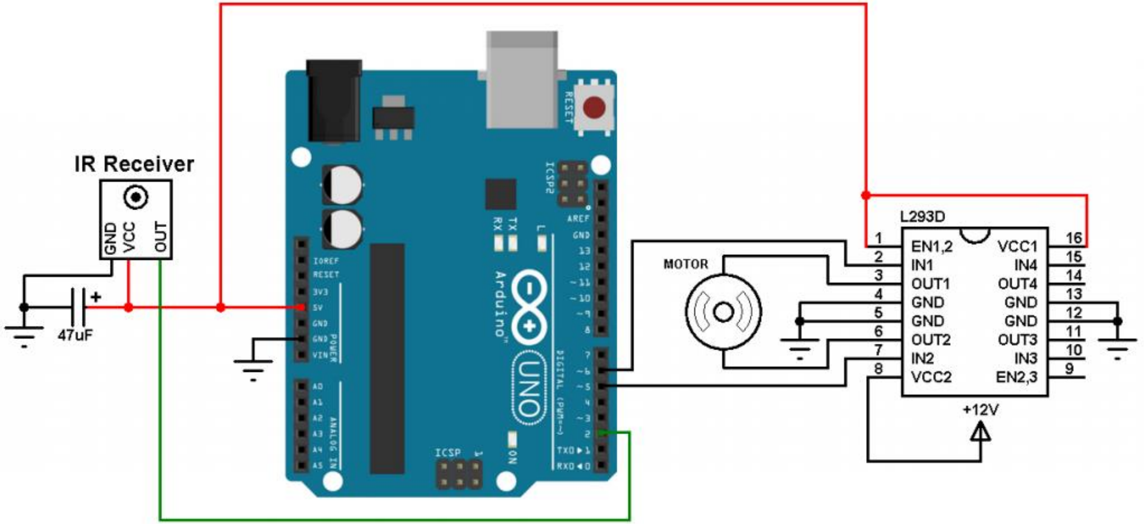
Programming

```
case third_key:
if(led[3] == 1)
{
digitalWrite(third_led_pin, LOW);
led[3] = 0;
}
else
{
digitalWrite(third_led_pin, HIGH);
led[3] = 1;
}
break;
```

```
case fourth_key:
if(led[4] == 1)
 {
digitalWrite(fourth_led_pin, LOW);
 led[4] = 0;
}
else
 {
digitalWrite(fourth_led_pin, HIGH);
 led[4] = 1;
}
break;
 }
Serial.println(value);
receiver.resume();
}
}
```

DC Motor Remote Control

Components Required:
- Arduino UNO board
- L293D motor driver IC
- 12V DC motor
- IR remote control
- IR receiver
- 47 uF capacitor
- 12V source
- Breadboard
- Jumper wires



BRUTE FORCE **bruteforcehub@gmail.com** **+263718384192**

Programming

```
// Arduino remote controlled DC motor (speed and direction control)
// NEC IR remote control is used (Car MP3)

#define pwm1    5
#define pwm2    6

boolean nec_ok = 0, motor_dir = 0, repeated = 0;
byte  i, nec_state = 0, duty_cycle =  0;
unsigned long nec_code;

void setup() {
  pinMode(pwm1, OUTPUT);
  pinMode(pwm2, OUTPUT);
  // Timer1 module configuration
  TCCR1A = 0;
  TCCR1B = 0;                                  // Disable Timer1 module
  TCNT1  = 0;                                  // Set Timer1 preload value to 0 (
  TIMSK1 = 1;                                  // enable Timer1 overflow interrup
  attachInterrupt(0, remote_read, CHANGE);     // Enable external interrupt (INT0
}

void remote_read() {
  unsigned int timer_value;
  if(nec_state != 0){
    timer_value = TCNT1;                       // Store Timer1 value
    TCNT1 = 0;                                 // Reset Timer1
  }
```

Programming

```
switch(nec_state){
 case 0 :                                      // Start receiving IR data (we're at t
  TCNT1  = 0;                                  // Reset Timer1
  TCCR1B = 2;                                  // Enable Timer1 module with 1/8 pres
  nec_state = 1;                               // Next state: end of 9ms pulse (start
  i = 0;
  return;
 case 1 :                                      // End of 9ms pulse
  if((timer_value > 19000) || (timer_value < 17000)){          // Invalid interval ==
    nec_state = 0;                             // Reset decoding process
    TCCR1B = 0;                                // Disable Timer1 module
  }
  else
    nec_state = 2;                             // Next state: end of 4.5ms space (st
  return;
 case 2 :                                      // End of 4.5ms space
  if((timer_value > 10000) || (timer_value < 3000)){
    nec_state = 0;                             // Reset decoding process
    TCCR1B = 0;                                // Disable Timer1 module
  }
  else{
    nec_state = 3;                             // Next state: end of 562µs pulse (st
    if(timer_value < 6000)                     // Check if previous code is repeated
      repeated = 1;
  }
  return;
 case 3 :                                      // End of 562µs pulse
```

Programming

```
  if((timer_value > 1400) || (timer_value < 800)){           // Invalid interval ==>
    TCCR1B = 0;                                 // Disable Timer1 module
    nec_state = 0;                              // Reset decoding process
  }
  else {
    // Check if the repeated code is for button 2 or 3
    if(repeated && (nec_code == 0x40BFB04F || nec_code == 0x40BF708F)){
      repeated = 0;
      nec_ok = 1;                               // Decoding process is finished with s
      detachInterrupt(0);                       // Disable external interrupt (INT0)
    }
    else
      nec_state = 4;                            // Next state: end of 562µs or 1687µs
  }
  return;
 case 4 :                                       // End of 562µs or 1687µs space
  if((timer_value > 3600) || (timer_value < 800)){           // Time interval invali
    TCCR1B = 0;                                 // Disable Timer1 module
    nec_state = 0;                              // Reset decoding process
    return;
  }
  if( timer_value > 2000)                       // If space width > 1ms (short space)
    bitSet(nec_code, (31 - i));                 // Write 1 to bit (31 - i)
  else                                          // If space width < 1ms (long space)
    bitClear(nec_code, (31 - i));               // Write 0 to bit (31 - i)
  i++;
  if(i > 31){                                   // If all bits are received
```

Programming

```
      nec_ok = 1;                             // Decoding process OK
      detachInterrupt(0);                     // Disable external interrupt (INT0)
      return;
    }
    nec_state = 3;                            // Next state: end of 562µs pulse (s
  }
}

ISR(TIMER1_OVF_vect) {                        // Timer1 interrupt service routine (
  nec_state = 0;                              // Reset decoding process
  TCCR1B = 0;                                 // Disable Timer1 module
}

void loop() {
  if(nec_ok) {
    nec_ok = 0;                                 // Reset decoding process
    nec_state = 0;
    TCCR1B = 0;                                 // Disable Timer1 module
    if(nec_code == 0x40BF30CF){                 // If button 1 is pressed (change 
      motor_dir = !motor_dir;                   // Toggle direction variable
      if(motor_dir)
        digitalWrite(pwm2, 0);
      else
        digitalWrite(pwm1, 0);
    }
```

Programming

```
    if(nec_code == 0x40BF708F && duty_cycle < 255)  // If button 3 is pressed (incr
      duty_cycle++;
    if(nec_code == 0x40BFB04F && duty_cycle > 0)    // If button 2 is pressed (decr
      duty_cycle--;
    if(motor_dir)
      analogWrite(pwm1, duty_cycle);
    else
      analogWrite(pwm2, duty_cycle);
    attachInterrupt(0, remote_read, CHANGE);        // Enable external interrupt (I
  }
}
```

Servo Motor Remote Control

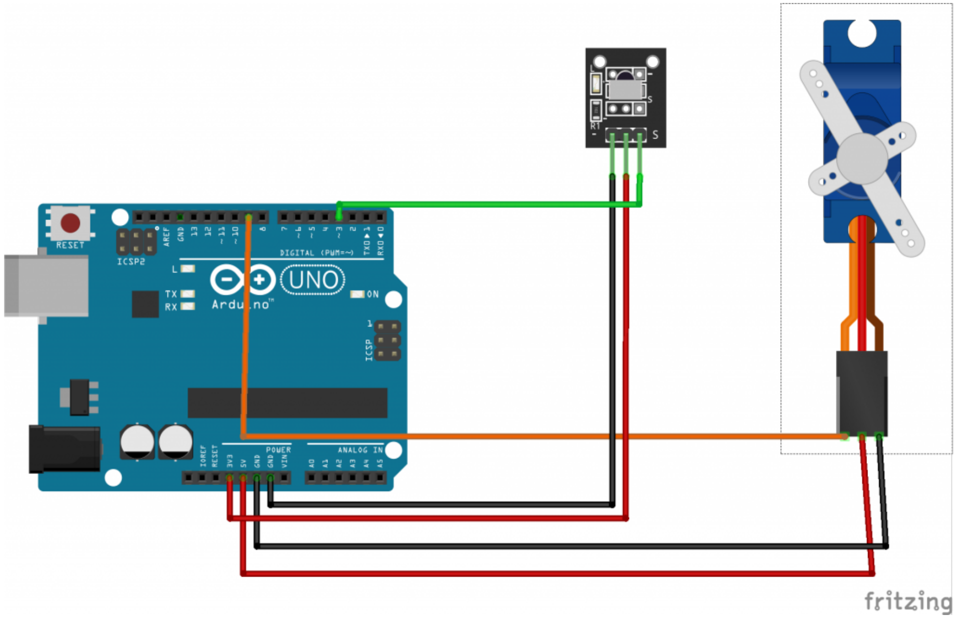
Preparations

HARDWARE

- Arduino UNO x 1
- Infrared Receiver x 1
- Remote Controller x 1
- SG90 Servo x 1
- Jumpers
- USB Cable x 1
- PC x 1

SOFTWARE

- Arduino IDE (version 1.6.4+)
- Arduino library: IRremote.h
- Arduino library: IRLibAll.h
- Arduino library: Servo.h



BRUTE FORCE **bruteforcehub@gmail.com** **+263718384192**

Servo Motor Remote Control

- Open IRremote demo example from Arduino IDE, then upload it to the Arduino board.
- Then open the serial monitor and try to click on any button on the remote control to send a signal to the IR receiver; the HEX code of each button must appear in the serial monitor
- Then detect the HEX code of the buttons used to control the servo motor; assume you will use two buttons of your choice, one for clockwise rotation and another for counter clockwise rotation.
- For example, use (▲) for clockwise and (▼ for counterclockwise, so you have to get their HEX codes.

•(▲) —> FF18E7
•(▼) —>FF4AB5

Programming

```
#include <IRremote.h>      //must copy IRremote library to arduino libraries
#include <Servo.h>
#define plus 0xFF18E7   //clockwise rotation button
#define minus 0xFF4AB5  //counter clockwise rotation button

int RECV_PIN = 3;       //IR receiver pin
Servo servo;
int val;                //rotation angle
bool cwRotation, ccwRotation;  //the states of rotation

IRrecv irrecv(RECV_PIN);

decode_results results;

void setup()
{
  Serial.begin(9600);
  irrecv.enableIRIn(); // Start the receiver
  servo.attach(9);     //servo pin
}
```

BRUTE FORCE bruteforcehub@gmail.com +263718384192

Programming

```
void loop()
{
 if (irrecv.decode(&results))
 {
Serial.println(results.value, HEX);
 irrecv.resume();  // Receive the next value
if (results.value == plus)
 {
cwRotation = !cwRotation;      //toggle the rotation value
   ccwRotation = false;         //no rotation in this direction
  }
 if (results.value == minus)
 {
ccwRotation = !ccwRotation;   //toggle the rotation value
  cwRotation = false;           //no rotation in this direction
 }
 }
 if (cwRotation && (val != 175))
 {
 val++;                         //for colockwise button
}
 if (ccwRotation && (val != 0))
{
val--;                         //for counter colockwise button
}  servo.write(val);
delay(20);         //General speed
 }
```

THANK YOU ARDUINO GEEKS!

BRUTE
FORCE
MAKERS SPACE

IGNITING CREATIVITY - LEARN BY DOING

bruteforcehub@gmail.com +263718384192